

Fizetések CRUD műveletei**Fizetés létrehozása:**

A fizetés létrehozása funkció fejlesztése során elkészítettem az új befizetések rögzítésére szolgáló API végpontot. A rendszer eltárolja a szerződés azonosítóját, a befizetett összeget, a fizetés dátumát, státuszát és fizetési módját. A fejlesztés során kialakítottam az adatok adatbázisba történő mentésének folyamatát. A végpont sikeres adatmentés esetén visszajelzést küld a kliens számára. A működés helyességét Postman alkalmazással ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    szerzodesid,
    osszeg,
    fizetesdatum,
    statusz,
    fizetesmod,
    megjegyzes
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO fizetesek
      (szerzodesid, osszeg, fizetesdatum, statusz, fizetesmod, megjegyzes)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [
        szerzodesid,
        osszeg,
        fizetesdatum,
        statusz,
        fizetesmod,
        megjegyzes
      ]
    );

    res.json({
      siker: true,
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        uzenet: "Fizetes sikeresen rogzitve!"
    });

    } catch (err) {

        res.status(500).json(err);

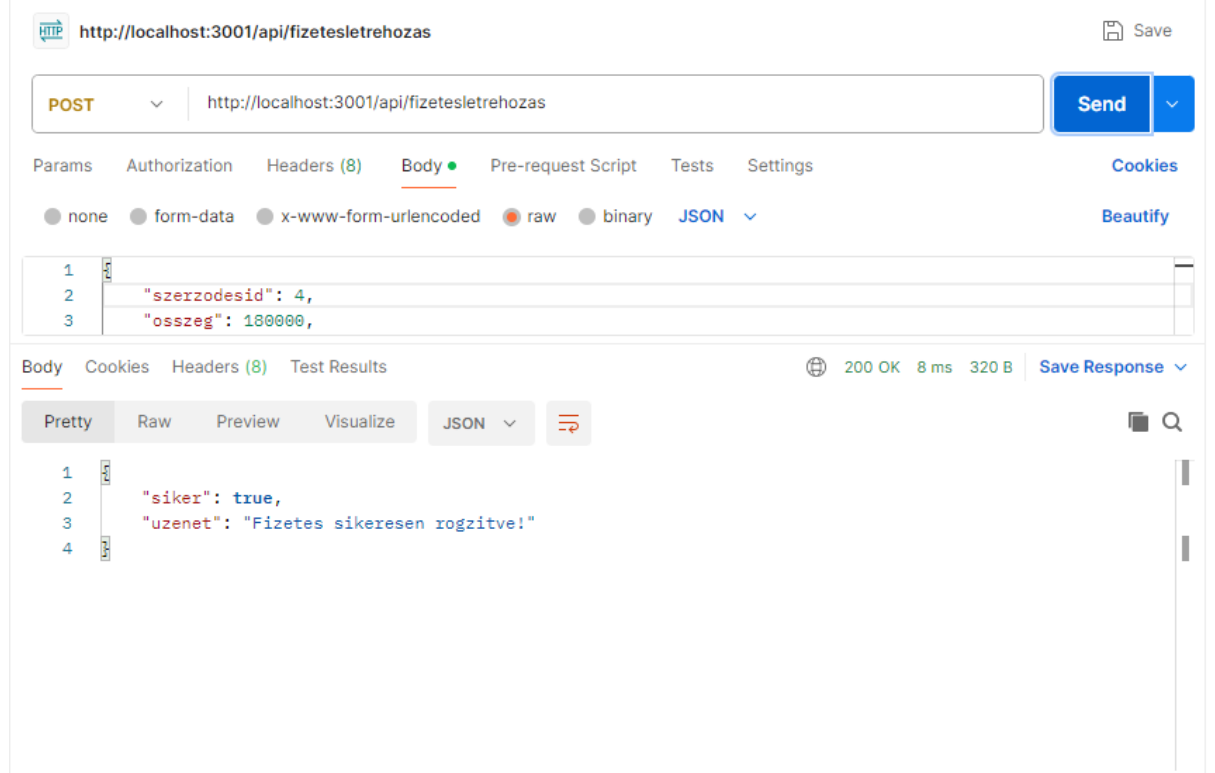
    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;
```



1. Részlet az fizetesletrehozaz.js fájlból.

Fizetés módosítása:

A fizetés módosítása funkció elkészítése során lehetővé tettem a már rögzített fizetési adatok szerkesztését. A rendszer képes módosítani a befizetett összeget, a fizetés dátumát, státuszát, fizetési módját és a megjegyzéseket. A fejlesztés során elkészítettem a szükséges SQL frissítő lekérdezéseket és backend logikát. A sikeres módosítás után a rendszer visszajelzést küld a felhasználó számára. A működés helyességét Postman segítségével ellenőriztem.

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    osszeg,
    fizetesdatum,
    statusz,
    fizetesmod,
    megjegyzes
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE fizetesek
      SET
        osszeg = ?,
        fizetesdatum = ?,
        statusz = ?,
        fizetesmod = ?,
        megjegyzes = ?
      WHERE id = ?`,
      [
        osszeg,
        fizetesdatum,
        statusz,
        fizetesmod,
        megjegyzes,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Fizetes modositva!"
    });

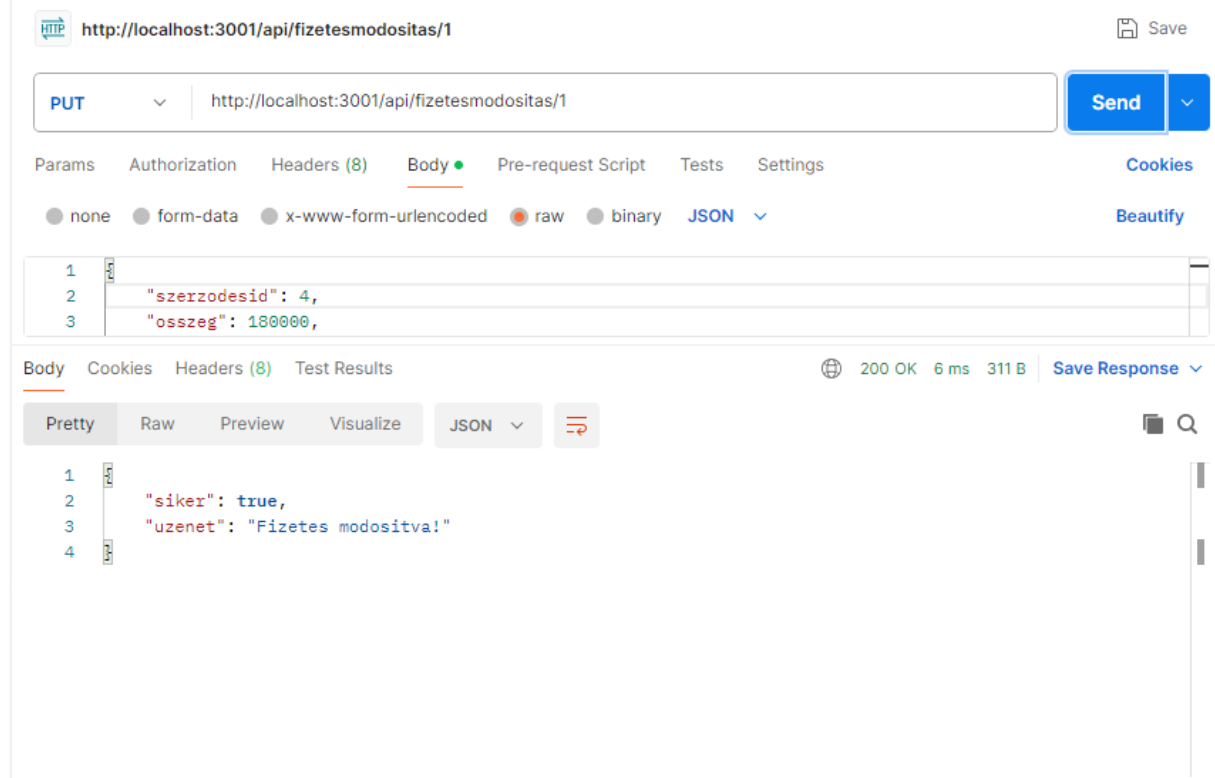
  } catch (err) {

    res.status(500).json(err);
  }
});
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
    } finally {  
  
        if(conn) conn.release();  
  
    }  
});  
  
module.exports = router;
```



2. Részlet az `fizetesmodositas.js` fájlból.

Fizetés törlése:

A fizetés törlése funkció fejlesztése során elkészítettem a fizetési rekordok eltávolítására szolgáló API végpontot. A rendszer a megadott azonosító alapján törli a kiválasztott befizetést az adatbázisból. A fejlesztés során biztosítottam a hibák megfelelő kezelését és az adatbázis-kapcsolat szabályos lezárását. A sikeres törlést követően a rendszer tájékoztató üzenetet küld vissza a kliens oldal számára. Az elkészült funkció működését Postman alkalmazás segítségével teszteltem és ellenőriztem.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.delete("/:id", async (req, res) => {
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
let conn;

try {

  conn = await pool.getConnection();

  await conn.query(
    "DELETE FROM fizetesek WHERE id = ?",
    [req.params.id]
  );

  res.json({
    siker: true,
    uzenet: "Fizetes torolve!"
  });

} catch (err) {

  res.status(500).json(err);

} finally {

  if(conn) conn.release();

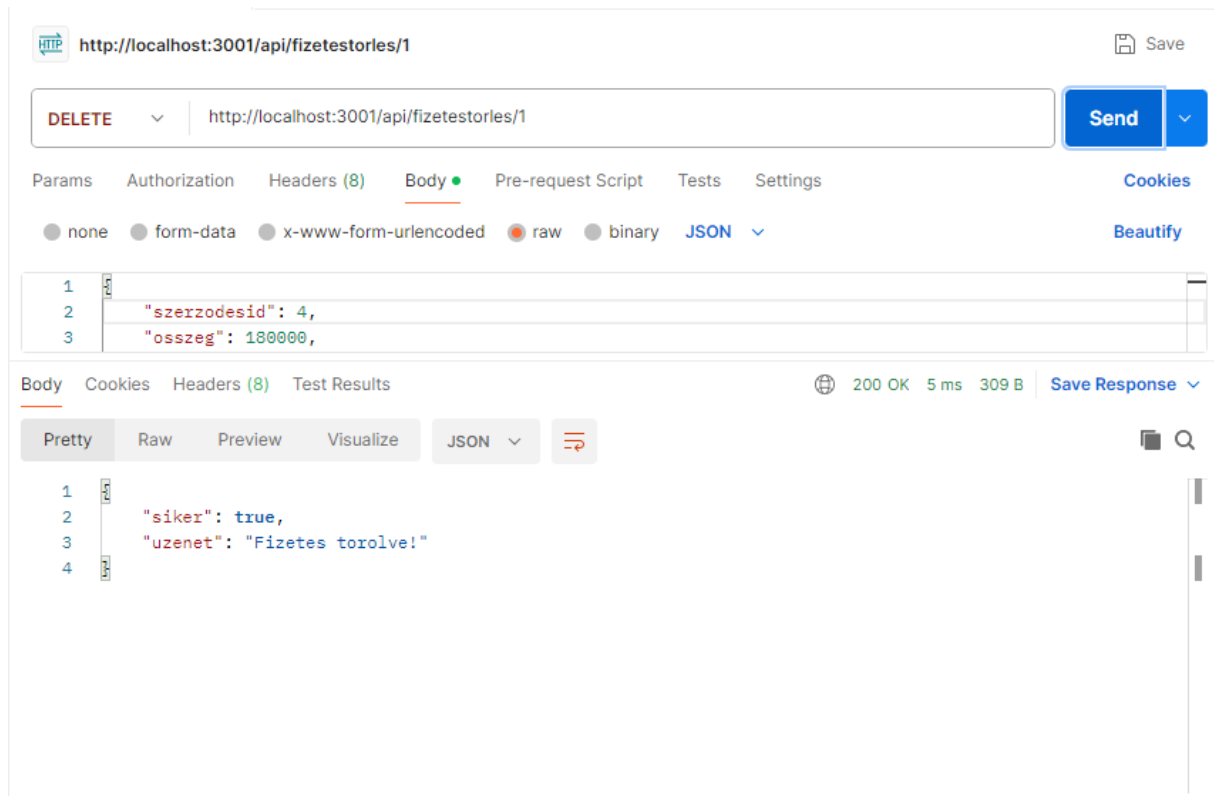
}

});

module.exports = router;
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9



3.Részlet az `fizetestorles.js` fájlból.

Fizetés listázása:

A fizetések listázása funkció fejlesztése során elkészítettem az összes befizetés lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a fizetésekhez tartozó adatokat, majd JSON formátumban visszaküldi azokat. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető formában láthassák a befizetések adatait. A lekérdezések során biztosítottam az adatbázis-kapcsolat megfelelő kezelését. Az elkészült funkciót Postman segítségével teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM fizetesek"
    );

    res.json(adatok);
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
    } catch (err) {  
  
        res.status(500).json(err);  
  
    } finally {  
  
        if(conn) conn.release();  
  
    }  
  
});  
  
module.exports = router;
```

The screenshot shows a REST client interface with the following details:

- URL: `http://localhost:3001/api/fizeteslistazas`
- Method: `GET`
- Response Status: `200 OK`
- Response Time: `6 ms`
- Response Size: `433 B`
- Response Body (JSON):

```
{  
  "id": 2,  
  "szerzodesid": 4,  
  "osszeg": "180000.00",  
  "fizetesdatum": "2026-01-09T23:00:00.000Z",  
  "statusz": "Befizetve",  
  "fizetesmod": "Atutalas",  
  "megjegyzes": "Januari lakber"  
}
```

4. Részlet az `fizeteslistazas.js` fájlból.